Experiment 14

Pro-Code Extension Using Zoho Catalyst

Aim

To study and implement pro-code extension using Zoho Catalyst to extend the Vehicle Service application with custom backend logic, APIs and database operations that go beyond the built-in capabilities of Zoho Creator.

Introduction

Zoho Catalyst is a pro-code development platform that enables developers to build, deploy and manage applications using programming languages and cloud-based services. It supports backend functions, data storage, API development, authentication and integrations.

Pro-code extension combines the rapid development of Zoho Creator with the flexibility of custom programming. For the Vehicle Service application, a Catalyst function can calculate a detailed service estimate (labour, parts and tax) that the Creator form calls through an API.

Procedure

1. Open the Zoho Catalyst website (catalyst.zoho.com) and click Access Catalyst to sign in with your Zoho account.
2. Create a new Catalyst project (for example, VehicleService-Project) and open its console.
3. Open CloudScale to view the available components: Authentication, Data Store, File Store, Web Client Hosting, API Gateway, Cron and others.
4. Review the Install CLI section. Install the Catalyst command-line tool using npm, log in with **catalyst login** and initialise the project with **catalyst init**.
5. Open the Data Store and create a table (for example, ServiceEstimate) to hold estimate data.
6. Review the table schema, which contains the system columns ROWID, CREATORID, CREATEDTIME and MODIFIEDTIME, and add columns such as Request_ID, Labour_Cost, Parts_Cost and Total_Estimate.
7. Use Catalyst serverless functions and the API Gateway to provide custom backend logic that the Zoho Creator application can call.

CLI Commands

```
npm install -g zcatalyst-cli
catalyst login
catalyst init
catalyst deploy
```

Data Store Table: ServiceEstimate

Column	Type	Purpose
ROWID, CREATORID, CREATEDTIME, MODIFIEDTIME	System	Created automatically for every table
Request_ID	Text	Service Request ID from Zoho Creator
Labour_Cost	Decimal	Labour charges
Parts_Cost	Decimal	Spare-parts charges
Total_Estimate	Decimal	Calculated estimate returned to Creator

Sample Function: Service Estimate Calculator (Node.js)

```
const express = require("express");
const app = express();
app.use(express.json());

app.post("/estimate", (req, res) => {
    const { labour, parts, discountPercent = 0 } = req.body;
    const subtotal = Number(labour) + Number(parts);
    const gst = subtotal * 0.18;
    const total = subtotal + gst - (subtotal * discountPercent) / 100;
    res.json({ subtotal, gst, total });
});

module.exports = app;
```

Process Diagram

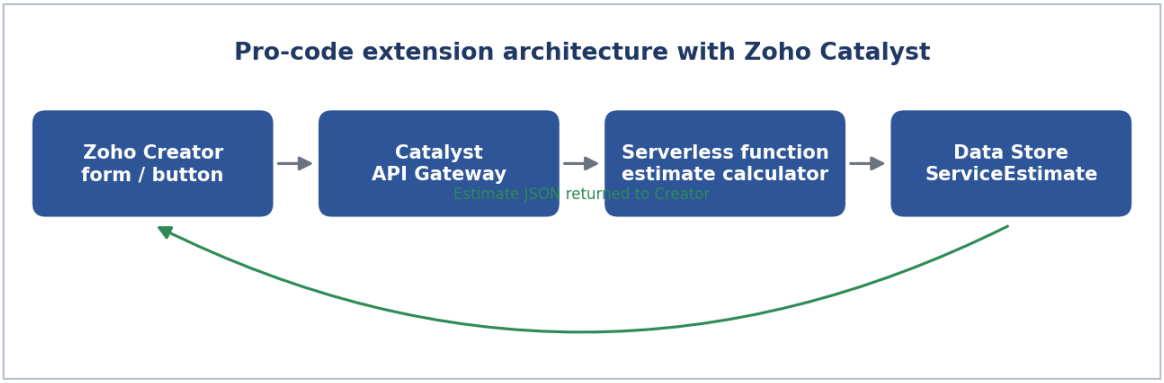


Figure 14.1: Diagram of the Catalyst extension architecture (illustrative diagram, not a screenshot).

Function Test (Expected)

Request body and the response calculated by the sample function (labour ₹1,500, parts ₹2,500, discount 10%):

```
POST /estimate
{ "labour": 1500, "parts": 2500, "discountPercent": 10 }

Response
{ "subtotal": 4000, "gst": 720, "total": 4320 }
```

Deployment Checklist

- Install the Catalyst CLI and log in.
- Initialise the project and add the function.
- Deploy with catalyst deploy and note the API Gateway URL.
- Call the URL from Deluge using invokeUrl and a connection.

Result

The Zoho Catalyst project and its CloudScale Data Store were set up, and the pro-code extension approach for extending the Vehicle Service application was studied.

Conclusion

Pro-code extension using Zoho Catalyst combines the rapid development benefits of low-code with the flexibility of traditional programming. It enables custom backend services, external integrations and advanced functionality beyond the standard features of Zoho Creator.